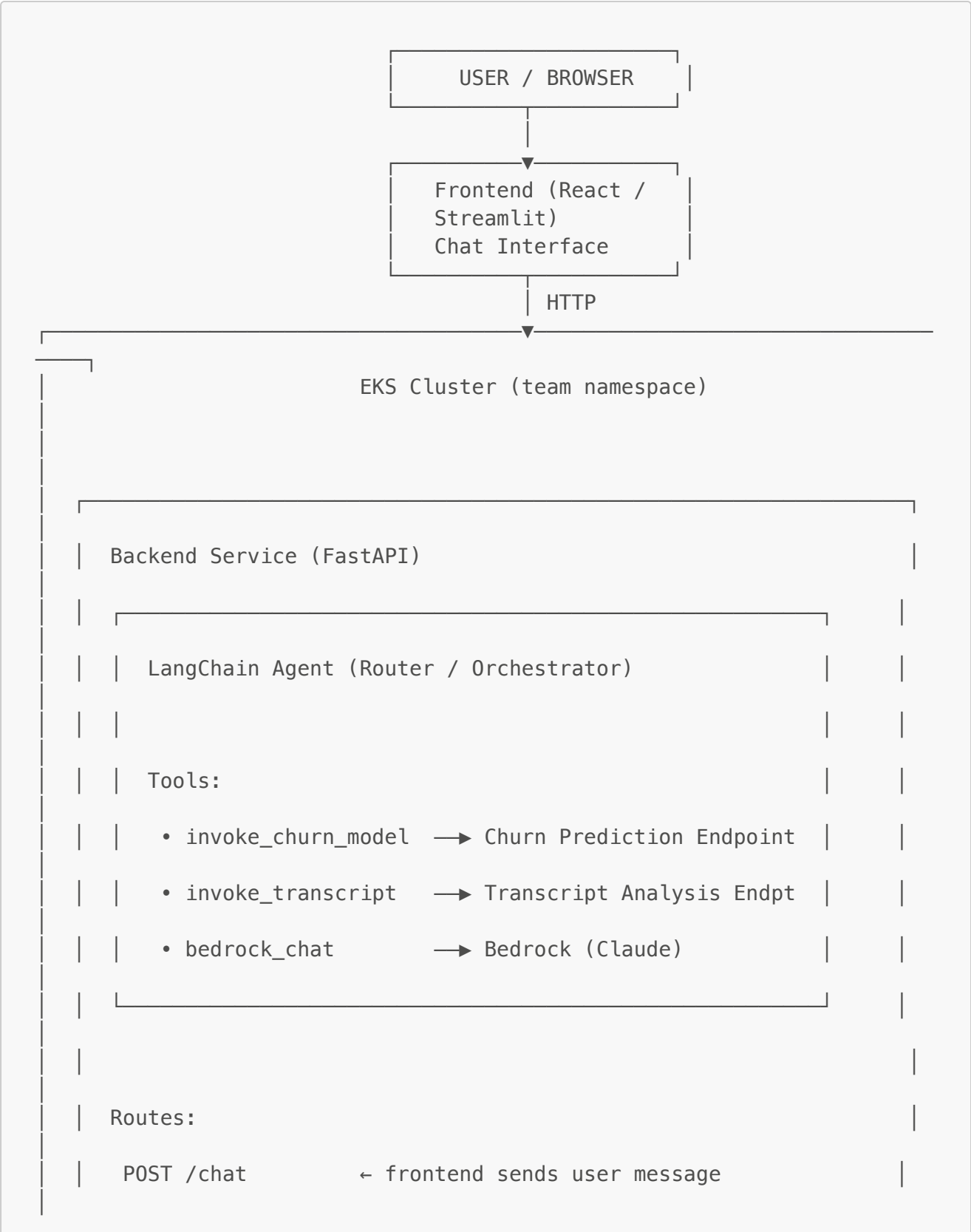
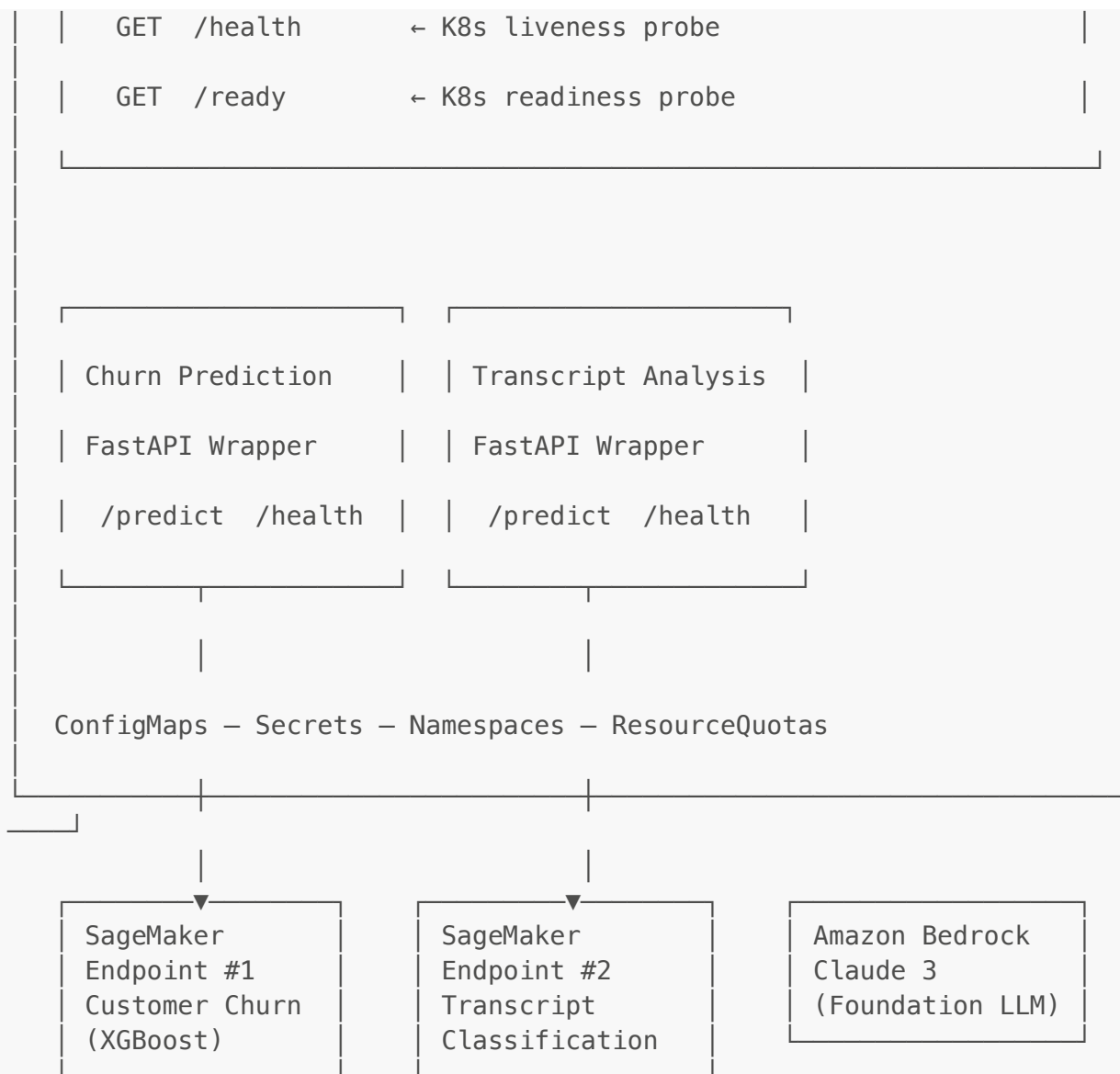


The Retention Engine — Project Roadmap

Team: Troy, Kathleen, Okino, George

Platform Architecture





INFRA: Terraform (S3, IAM, SageMaker, EKS references)
 CI/CD: GitHub Actions → Build → Push GHCR → Deploy to EKS

Repo Structure

```
capstone/
├── .github/
│   └── workflows/
│       ├── terraform.yml           # Terraform plan/apply
│       ├── sagemaker-deploy.yml    # Deploy/delete SageMaker
│       ├── deploy.yml              # Docker build → GHCR → EKS
│       ├── ci-post-merge.yml       # Auto post-merge: tests → Docker
│       └── slack-pr-events.yml     # Slack notifications on PR
├── (workflow_dispatch)
├── endpoints (workflow_dispatch)
├── rollout (workflow_dispatch)
└── → endpoints → E2E
```

```

open/merge
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── providers.tf
│   └── modules/                                # (bonus) networking, iam, sagemaker
├── k8s/
│   ├── namespace.yaml
│   ├── configmaps/
│   │   └── agent-config.yaml
│   ├── deployments/
│   │   ├── agent-deployment.yaml
│   │   ├── backend-deployment.yaml
│   │   ├── churn-wrapper-deployment.yaml
│   │   ├── frontend-deployment.yaml
│   │   └── transcript-wrapper-deployment.yaml
│   └── secrets.yaml                            # (template – real values via GH
Secrets)
├── services/
│   ├── backend-service.yaml
│   ├── churn-wrapper-service.yaml
│   ├── transcript-wrapper-service.yaml
│   └── frontend-service.yaml
├── backend/
│   ├── Dockerfile
│   └── app/
│       └── main.py                            # FastAPI app, /chat, /health,
/ready
│   ├── agent.py                              # LangChain agent + tool definitions
│   ├── tools/
│   │   ├── churn_tool.py                    # Calls churn-wrapper /predict
│   │   ├── transcript_tool.py               # Calls transcript-wrapper /predict
│   │   └── bedrock_tool.py                  # Bedrock invoke for general chat
│   └── config.py                             # Env var config
├── requirements.txt
├── ml-wrappers/
│   ├── churn/
│   │   ├── Dockerfile
│   │   ├── app.py                           # FastAPI /predict + /health
│   │   └── requirements.txt
│   └── transcript/
│       ├── Dockerfile
│       ├── app.py                           # FastAPI /predict + /health
│       └── requirements.txt
├── sagemaker/
│   ├── churn/
│   │   ├── train.py                         # Training script or notebook
│   │   ├── deploy.py                       # Endpoint creation script
│   │   └── data/                           # Sample data / data prep scripts
│   └── transcript/
│       ├── train.py
│       └── deploy.py

```

```

├── data/
├── frontend/
│   ├── Dockerfile
│   ├── src/
│   └── ...
├── docker-compose.yaml
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   ├── deployment.md
│   ├── teardown.md
│   └── llm-usage/
├── PROJECT_ROADMAP.md
└── README.md
# React or Streamlit app
# Local dev: all services together
# Per-member LLM usage docs

```

Owner Matrix

Component	Primary Owner	Support	Key Deliverables
GitHub Projects / Kanban	Troy	All	Board setup, task cards, sprint tracking
GitHub Actions CI/CD	Troy	George	All .yaml workflows, secrets config
Dataset identification	Kathleen + Okino	—	Churn dataset, transcript dataset
SageMaker training + deploy	Kathleen + Okino	—	sagemaker/ training scripts, live endpoints
ML FastAPI wrappers	Kathleen + Okino	Troy (CI)	ml-wrappers/ services with /predict + /health
Terraform infra	George	Troy (CI)	terraform/ — IAM, S3, SageMaker resources
Kubernetes manifests	George	Troy (deploy wf)	k8s/ — all manifests
LangChain agent + Bedrock	George	Kathleen/Okino (tool contracts)	backend/app/agent.py , tool definitions
Backend FastAPI service	George	All	backend/ — routes, agent wiring
Frontend chat UI	All	—	frontend/ — chat interface
Documentation + diagrams	All	—	docs/ , README.md

Graded Areas → Owner Mapping

#	Graded Area (10% each)	Primary Owner	Supporting
1	Containers & Dockerfiles	George (backend, frontend), Kathleen/Okino (ML wrappers)	Troy (CI builds)
2	Terraform Provisioning	George	—
3	CI/CD with GitHub Actions	Troy	All contribute workflows
4	LangChain Agentic Harness	Kathleen (LangGraph + LangSmith), George (initial scaffold)	Okino (tool contracts)
5	Bedrock Chat Agent & Frontend	George (Bedrock/agent), All (frontend)	—
6	SageMaker ML Endpoints	Okino (Endpoint 1: QA/Transcript), Kathleen (Endpoint 2: Churn)	George (integration)
7	Kubernetes Orchestration	George	Troy (CI/CD deploy steps)
8	Team Collaboration & Agile	Troy (board setup), All (participation)	—
9	Presentation & Documentation	All	—
10	Leveraging LLMs as Dev Tools	All (individual docs)	—

Interface Contracts

Define these early so everyone can work in parallel.

Churn Wrapper → SageMaker Endpoint

```
POST /predict
Request: { "features": [0.5, 1.0, 3, 45.2, ...] }
Response: { "churn_probability": 0.82, "prediction": "churn" }

GET /health
Response: { "status": "healthy", "endpoint": "churn-endpoint-v1" }
```

Transcript Wrapper → SageMaker Endpoint

```
POST /predict
Request: { "text": "I've been waiting for 30 minutes and nobody..." }
Response: { "category": "complaint", "sentiment": "negative",
"confidence": 0.91 }

GET /health
Response: { "status": "healthy", "endpoint": "transcript-endpoint-v1" }
```

Backend /chat → Frontend

```
POST /chat
Request: { "message": "What's the churn risk for customer 4521?" }
Response: { "response": "Based on the model prediction, customer 4521
has a 82% churn probability..." }

GET /health
Response: { "status": "healthy", "agent": "ready", "tools": ["churn",
"transcript", "bedrock"] }
```

LangChain Agent Design

```
# Simplified agent structure – George owns, Kathleen/Okino define tool
I/O

tools = [
    # Tool 1: Churn prediction
    #   Input: customer feature vector
    #   Action: POST to churn-wrapper-service:8000/predict
    #   Output: churn probability + label

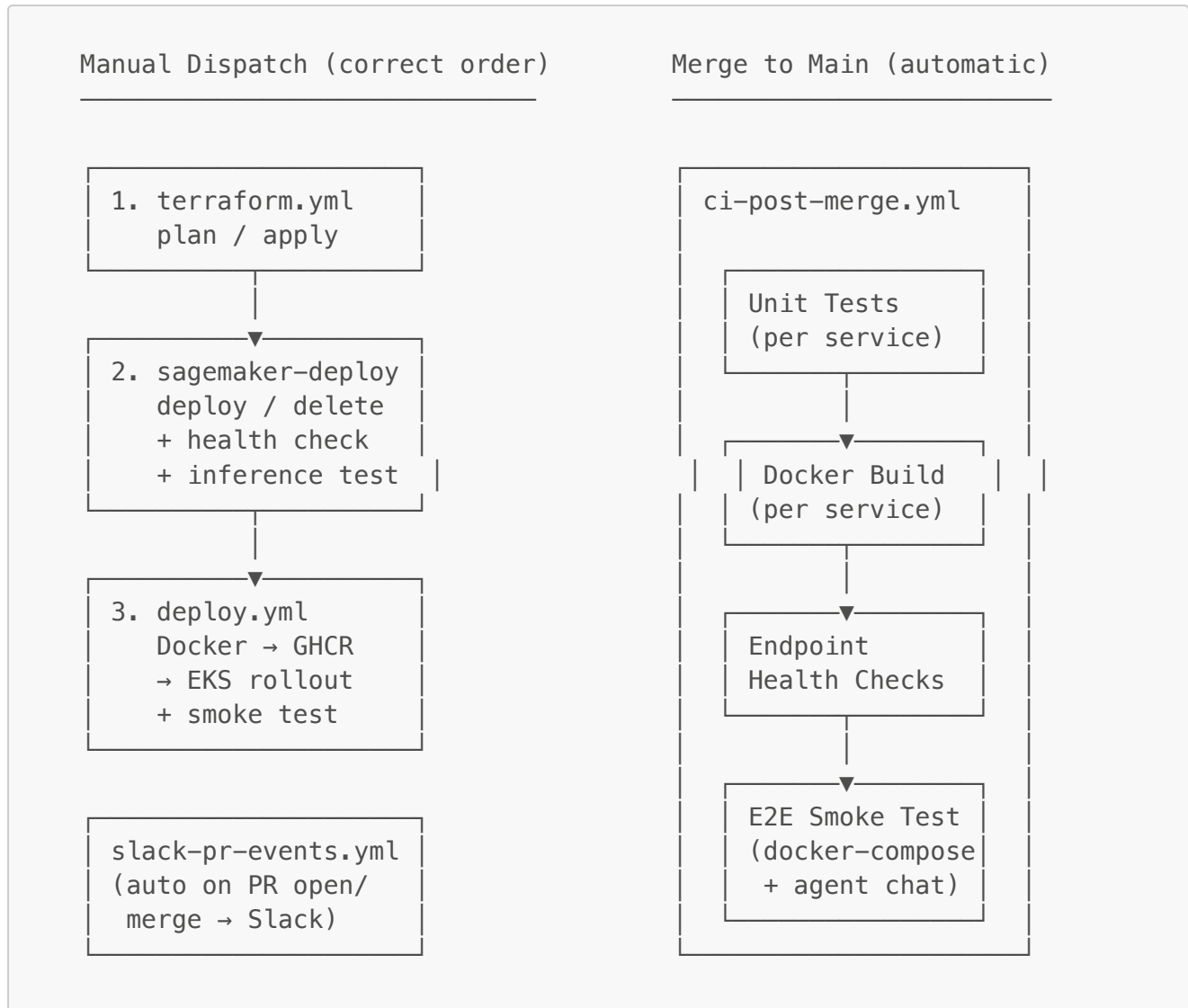
    # Tool 2: Transcript analysis
    #   Input: raw text from service call
    #   Action: POST to transcript-wrapper-service:8000/predict
    #   Output: category + sentiment + confidence

    # Tool 3: Bedrock general chat
    #   Input: user message (when no tool needed)
    #   Action: bedrock invoke_model (Claude 3)
    #   Output: conversational response
]

# Agent routes based on user intent:
#   "Will customer X churn?"      → churn tool
#   "Analyze this transcript..." → transcript tool
#   "What can you help with?"    → bedrock general chat
#   Complex queries               → chain multiple tools
```

```
# Non-trivial agentic behavior:
#   If churn_probability > 0.7 → auto-generate retention offer via
Bedrock
#   Multi-step: transcript → QA score → combine with account data →
churn → retention offer
```

CI/CD Pipeline Overview



Workflows (Troy owns):

File	Trigger	What It Does
terraform.yml	workflow_dispatch (plan/apply, dev/staging/prod)	Terraform init → validate → plan → apply for S3, IAM, SageMaker
sagemaker-deploy.yml	workflow_dispatch (both/churn/sentiment, deploy/delete)	Deploy endpoints, poll for InService, validate inference

File	Trigger	What It Does
<code>deploy.yml</code>	<code>workflow_dispatch</code> (all/per-service, custom tag)	Matrix Docker build → GHCR push → kubectl apply → rollout restart → smoke test
<code>ci-post-merge.yml</code>	push to main	Unit tests → Docker build check → SageMaker health → E2E smoke test (docker-compose)
<code>slack-pr-events.yml</code>	PR opened / closed	Sends formatted Slack notification via webhook

Kubernetes Layout

```

Namespace: team-retention-engine
├── backend-deployment      (2 replicas)
│   └── backend-service     (ClusterIP or LoadBalancer)
├── churn-wrapper-deployment (1 replica)
│   └── churn-wrapper-service (ClusterIP)
├── transcript-wrapper-deployment (1 replica)
│   └── transcript-wrapper-service (ClusterIP)
├── frontend-deployment     (1 replica)
│   └── frontend-service    (LoadBalancer → public)
├── configmap               (endpoint URLs, region, model names)
└── secret                  (AWS creds – injected via GH Actions)

```

All inter-service calls stay **inside the cluster** (ClusterIP). Only the frontend service gets a LoadBalancer for public access. The backend talks to ML wrappers via K8s DNS (`churn-wrapper-service.team-retention-engine.svc.cluster.local:8000`).

Phase Overview

Presentation Date: April 23, 2026

Phase 0: Setup & Scaffolding		Mar 31 – Apr 1
Phase 1: Core Services		Apr 2 – Apr 8
Phase 2: Infra, Deploy & CI/CD		Apr 9 – Apr 15
Phase 3: Frontend, Polish & Docs		Apr 16 – Apr 22
Presentation Day		Apr 23

Phase 0 — Setup & Scaffolding (Mar 31 – Apr 1)  COMPLETE

- ✓ Kanban board, branch protection, workflow stubs, docker-compose
 - ✓ Terraform, K8s manifests, agent service scaffold, Dockerfiles, Bedrock access
 - ✓ TriLink transcript dataset, sentiment-analysis-api scaffold, training notebook
 - ✓ TriLink churn dataset, sagemaker/churn scaffold, churn-predictor-api scaffold
 - ✓ API contracts agreed, React + Vite + Tailwind chosen, LLM usage logging started
-

Phase 1 — Core Services (Apr 2 – Apr 8) ✓ MOSTLY COMPLETE

Troy — CI/CD Pipelines ✓

- ✓ All 5 workflow files implemented (terraform, sagemaker-deploy, deploy, ci-post-merge, slack-pr-events)
- ✓ Manual dispatch for infra/deploy with correct execution order
- ✓ Automatic post-merge validation with unit tests, Docker builds, endpoint health, E2E

Okino — QA/Transcript Evaluator

- ✓ Training notebook ([services/sentiment-analysis-api/sentiment_training.ipynb](#))
- ✓ FastAPI wrapper with Bedrock-powered sentiment analysis
- ✓ Dockerfile, K8s deployment + service manifests
- ✓ Terraform [sagemaker.tf](#) for endpoint config
- ☐ SageMaker endpoint deployment — in progress
- ☐ [/predict](#) route must return: qa_score, sentiment, emotion_frustration, emotion_anger, sentiment_shift, escalation_flag, resolution_flag

Kathleen — Churn Predictor ✓

- ✓ XGBoost model v3: 95% accuracy, 0.9861 AUC, 31 features
- ✓ Cross-model integration: 7 Agent 1 features from synthetic call data
- ✓ SageMaker endpoint deployed (XGBoost container, native format) — InService
- ✓ FastAPI wrapper with internal customer data lookup from S3
- ✓ [/predict](#) accepts customer_id + Agent 1 fields, looks up account data internally
- ✓ [/customers](#) API for frontend searchable dropdown
- ✓ [/customer-details/{id}](#) API for agent tool
- ✓ [/high-risk](#) API with batch SageMaker prediction + caching
- ✓ Dockerfile (multi-stage build, slim base, health check)

Kathleen & Okino — Orchestration ✓

- ✓ retention_agent.py: 4-tool agent (get_customer_details, analyze_call, predict_churn, get_high_risk_customers)
- ✓ TriLink product catalog with approved retention actions per risk level
- ✓ Output guardrails validating agent recommendations
- ✓ Conversation memory (InMemoryChatMessageHistory, session-based)

- ☒ Intelligent routing: agent decides tools based on question type
- ☒ churn_tool.py: sends customer_id + 7 Agent 1 fields
- ☒ high_risk_tool.py: fetches ranked at-risk customer list
- ☒ customer_tool.py: looks up account details for conversational queries

George — Infrastructure & Agent

- ☒ Terraform: S3, IAM roles, sagemaker.tf, iam.tf, s3.tf
- ☒ K8s: all deployments, services, configmaps, secrets, namespace quota
- ☒ Agent service: app.py with /chat route, CORS, session_id support
- ☒ docker-compose.yml for full local stack
- ☐ Deploy full stack to EKS

Kathleen — Frontend ☒

- ☒ React + Vite + Tailwind Manager's Command Center
- ☒ **Analyze tab**: searchable customer dropdown, RiskCard, SentimentCard, ActionCard
- ☒ **Chat tab**: conversational Bedrock interface with smart fallback
- ☒ Lucide React SVG icons (professional)
- ☒ Dockerfile (multi-stage: node → nginx)

Phase 2 — Infrastructure, Deploy & CI/CD (Apr 9 – Apr 15)

Troy — Deploy Pipeline ☒

- ☒ terraform.yml — plan/apply with environment selection (dev/staging/prod)
- ☒ sagemaker-deploy.yml — deploy/delete endpoints with health polling + inference validation
- ☒ deploy.yml — matrix build → GHCR → EKS rollout with smoke test
- ☒ ci-post-merge.yml — 4-job automatic pipeline on push to main
- ☒ slack-pr-events.yml — Slack notifications on PR open/merge

Okino — Finish Endpoint 1

- ☐ Deploy sentiment model to SageMaker endpoint
- ☐ Confirm /predict output matches churn_tool.py expected fields
- ☐ Test end-to-end: transcript → Agent 1 → Agent 2 → Agent 3

George — K8s Deploy

- ☐ Deploy full stack to EKS
- ☐ Verify Bedrock + SageMaker access from inside cluster
- ☐ Test full agent pipeline from deployed frontend

Kathleen & Okino — Integration Testing

- ☐ Test full 3-agent pipeline end-to-end with live endpoints
- ☐ Verify high-risk batch prediction works on EKS

Phase 3 — Frontend, Polish & Docs (Apr 16 – Apr 22)

Frontend — Manager's Command Center

- ☒ Analyze tab with customer dropdown + results panel
- ☒ Chat tab with Bedrock conversational interface
- ☒ Lucide icons, Tailwind styling
- ☐ "High Risk" leaderboard dashboard panel (bonus)

Memory Enhancement (Bonus — if time allows)

- ☐ Integrate mem0 for persistent semantic memory (reference: CyberRisk portfolio project)
- ☐ Requires: PostgreSQL + pgvector (could use AWS RDS)
- ☐ Would enable: long-term user preferences, cross-session context, semantic search over past analyses
- ☐ Alternative: current InMemoryChatMessageHistory is sufficient for demo

Troy — Final CI/CD & Collaboration

- ☐ Wire frontend CI/CD
- ☐ Final Kanban cleanup
- ☐ Write docs: CI/CD setup guide, teardown steps

Kathleen & Okino — ML Docs & Bonus

- ☒ Kathleen's SageMaker endpoint verified InService
- ☒ Kathleen LLM usage documented ([docs/llm-usage/kathleen-llm-usage.md](#))
- ☐ Okino's SageMaker endpoint — needs deployment
- ☐ Okino LLM usage doc
- ☐ Ensure model invocations work from inside K8s pods

George — Infrastructure Finalization

- ☐ Verify Terraform state is clean and reproducible
- ☐ Terraform remote state with S3/DynamoDB (bonus)
- ☐ Architecture diagram
- ☐ Write docs: setup, deployment, teardown steps

Bonus Integrations (Anyone)

- ☐ Amazon Transcribe pipeline: S3 audio → Transcribe → feed to agent
- ☐ Amazon Polly: text-to-speech for agent responses
- ☐ Model drift tracking — lightweight [/drift](#) endpoint in churn predictor:
 - Log prediction distributions over time (rolling window)
 - Compare incoming feature distributions to training baselines
 - Return drift metrics (mean shift, distribution divergence)
 - Alert when predictions skew beyond threshold
 - No SageMaker Model Monitor needed — runs inside the FastAPI wrapper

Documentation (All Members)

- ☐ **README.md** — project overview, team members, quick start
- ☐ **docs/setup.md** — full reproduction steps (clone → provision → deploy → test → teardown)
- ☐ **docs/architecture.md** — at least one architecture diagram:
 - System-level diagram showing all services and data flow
 - Deployment pipeline diagram
 - (Bonus) C4 diagrams, sequence diagrams
- ☐ **docs/llm-usage/** — each member documents:
 - Which LLM tools they used
 - Specific examples of code generation, debugging, architecture planning
 - Critical evaluation: what they accepted, rejected, or modified
 - (Bonus) comparison of LLM-generated vs hand-written code

Presentation Prep

- ☐ Assign presentation sections — each member covers their area:
 1. **Business Problem & Architecture Overview** — any member (2 min)
 2. **Infrastructure & Terraform** — George (3 min)
 3. **CI/CD & Git Workflow** — Troy (3 min)
 4. **ML Endpoints & Data Pipeline** — Kathleen & Okino (3 min)
 5. **Agentic Layer, Bedrock, K8s & Frontend** — George (3 min)
 6. **Live Demo** — all (3 min)
 7. **LLM Usage & Lessons Learned** — all (2 min)
- ☐ Rehearse at least once as a team
- ☐ Prepare for Q&A on any section

Final Checks

- ☐ GitHub Projects board is up to date with task history
- ☐ All PRs have descriptions and at least one review
- ☐ Commit history shows contributions from all members
- ☐ No AWS credentials in the repo (scan with `git log --all -p | grep -i "AKIA"`)
- ☐ All services healthy on EKS
- ☐ CI/CD pipelines green

Milestone ✓

Demo-ready. Clone → follow docs → full deployment reproducible.

Parallel Work Strategy

The key to speed: **everyone works simultaneously from Day 1.**

Troy

Kathleen/Okino

George

Week 1 Focus:	CI/CD Repo Workflows	ML Models Datasets SageMaker Train FastAPI Wrappers	Infra + Agent Terraform K8s Manifests LangChain
Week 2 Focus:	Frontend Final CI Docs	Frontend Hardening Docs	Frontend Deploy to EKS Docs

Integration points (where people need to sync):

1. **API contracts** — agree on request/response shapes before coding (Day 1)
2. **Docker image names** — Troy needs image paths for CI, George needs them for K8s
3. **Endpoint URLs** — Kathleen/Okino provide SageMaker endpoint names, George puts them in ConfigMaps
4. **Secrets** — Troy sets up GH Secrets, George references them in K8s Secrets

Bonus Points Checklist

Area	Bonus Item	Difficulty	Owner	Status
Containers	Alpine/slim bases, health checks, <code>.dockerignore</code>	Low	All	✅ Done
Terraform	Remote state with S3/DynamoDB	Low	George	Pending
Terraform	Modular structure (separate modules)	Medium	George	Pending
CI/CD	Separate CI workflows per service	Low	Troy	✅ Done — matrix builds in <code>deploy.yml</code> + <code>ci-post-merge.yml</code>
CI/CD	Branch targeting, rollback, multi-workflow chaining	Medium	Troy	✅ Done — <code>workflow_dispatch</code> ordering, post-merge auto-validation
LangChain	LangGraph workflows	Medium	Kathleen	✅ Done — <code>retention_graph.py</code>
LangChain	LangSmith observability	Medium	Kathleen	✅ Done — retention-engine project
Bedrock/Chat	Conversation memory persistence	Medium	Kathleen	✅ InMemory done, mem0 stretch
Bedrock/Chat	Polished UI (Tailwind/MUI)	Medium	Kathleen	✅ Tailwind + Lucide

Area	Bonus Item	Difficulty	Owner	Status
SageMaker	Third endpoint or model versioning	Medium	Kathleen/Okino	Pending
SageMaker	Retry logic, error handling, fallback	Medium	Kathleen	✅ Fallback in chat
K8s	ResourceQuota + LimitRange	Low	George	✅ Done
K8s	HPA, persistent storage, rolling updates	Medium	George	Pending
Collaboration	Sprint artifacts, retro notes, standup docs	Low	Troy + All	Pending
Collaboration	Code review comments on PRs	Low	All	✅ Done (40+ PRs)
Presentation	C4/sequence diagrams	Medium	All	Pending
Presentation	Present early	Low	All	Pending
LLM Usage	Compare LLM vs hand-written code	Medium	Kathleen	✅ In llm-usage doc
LLM Usage	Document prompt engineering techniques	Low	Kathleen	✅ In llm-usage doc
Extra	AWS Transcribe/Polly integration	Medium	Anyone	Pending
Extra	Blog articles, MkDocs, video series	High	Anyone	Pending
Extra	Portfolio page integration	High	Anyone	Pending

Daily Standup Template

```

**Name:**
**Yesterday:** What I completed
**Today:** What I'm working on
**Blockers:** Anything stopping progress
**LLM Note:** How I used LLM tools today (for documentation)

```

Key Principles

1. **Vertical slice first** — get one path working end-to-end before expanding

2. **Parallel lanes** — infrastructure and application work happen simultaneously
3. **API contracts on Day 1** — agree on request/response shapes so everyone can work independently
4. **Feature branches always** — never commit directly to **main**
5. **Document as you go** — don't leave it all for Phase 3
6. **Log your LLM usage** — it's 10% of the grade, treat it seriously
7. **Test locally before deploying** — Docker Compose and Minikube first, EKS second
8. **Reuse prior work** — FastAPI services, K8s manifests, Terraform configs from earlier modules are fair game